JUNKIO EXPENSE TRACKER

Báo cáo thuyết minh kỹ thuật

Phiên bản tài liệu tái cấu trúc từ workspace docx

Ngày tạo: 10/04/2026

Mục lục

- 1. Mô tả bài toán và phạm vi
- 2. Biểu đồ Use Case
- 3. Kiến trúc hệ thống
- 4. Hình ảnh giao diện
- 5. Hướng dẫn sử dụng
- 6. Tài liệu tham chiếu
- 7. Tài liệu legacy

Mô tả bài toán

Bối cảnh

Quản lý tài chính cá nhân và gia đình thường bị phân tán trên nhiều kênh như ghi chú thủ công, ứng dụng ngân hàng, ví điện tử và các bảng tính rời rạc. Khi dữ liệu thu chi không được tập trung, người dùng khó nắm được dòng tiền thực tế, khó đánh giá mức độ tuân thủ ngân sách, và càng khó kiểm soát các khoản chi dùng chung trong gia đình.

Trong bối cảnh đó, một hệ thống quản lý chi tiêu cần giải quyết đồng thời ba nhu cầu: ghi nhận giao dịch nhanh và rõ ràng, theo dõi mục tiêu tài chính theo thời gian, và hỗ trợ cộng tác trong các quỹ hoặc nhóm gia đình mà vẫn giữ được tính minh bạch và phân quyền.

Bài toán cần giải quyết

Hệ thống Junkio Expense Tracker được xây dựng để giải quyết các nhóm vấn đề chính sau:

Nhóm vấn đề	Mô tả thực tế	Hệ quả
Dữ liệu phân tán	Giao dịch nằm ở nhiều ví, tài khoản, hóa đơn và ghi chú cá nhân	Không có bức tranh tổng thể về thu chi
Thiếu kiểm soát ngân sách	Người dùng biết mình có chi tiêu nhiều nhưng không biết vượt ở danh mục nào	Dễ vượt kế hoạch tài chính
Thiếu công cụ lập mục tiêu	Tiết kiệm thường không gắn với deadline hoặc mốc theo dõi cụ thể	Mục tiêu tài chính khó hoàn thành
Khó quản lý chi tiêu chung	Nhiều khoản chi phát sinh trong gia đình hoặc nhóm, nhưng không rõ ai góp và ai đang dùng quỹ nào	Thiếu minh bạch, khó đối soát
Thiếu góc nhìn phân tích	Người dùng chỉ thấy danh sách giao dịch, không thấy xu hướng và dự báo	Khó ra quyết định tài chính

Đối tượng sử dụng

• Khách chưa đăng nhập: đăng ký tài khoản, đăng nhập, yêu cầu đặt lại mật khẩu.

• Người dùng thường: quản lý ví, giao dịch, mục tiêu, gia đình, báo cáo, dự báo, hồ sơ và cài đặt.

• Admin hệ thống: theo dõi toàn cảnh nền tảng, quản lý người dùng, xem thống kê và nhật ký hoạt động.

Vai trò staff hiện có trong dữ liệu và phân quyền quản trị, nhưng chưa có luồng nghiệp vụ riêng đủ khác biệt để mô tả như một actor độc lập trong phạm vi báo cáo này.

Mục tiêu của hệ thống

• Tập trung dữ liệu thu chi cá nhân và gia đình vào một hệ thống thống nhất.

• Theo dõi giao dịch theo ví, loại giao dịch và bối cảnh sử dụng.

• Quản lý ngân sách và mục tiêu tiết kiệm theo thời gian.

• Hỗ trợ quỹ gia đình và các khoản chi dùng chung.

• Cung cấp báo cáo trực quan, phân tích xu hướng và dự báo.

• Hỗ trợ quản trị hệ thống ở mức nền tảng cho admin.

Giá trị mang lại

• Giúp người dùng thấy ngay tình hình tài chính hiện tại thay vì phải tổng hợp thủ công.

• Tăng khả năng kiểm soát chi tiêu và hạn chế vượt ngân sách.

• Hỗ trợ ra quyết định tài chính dựa trên dữ liệu và biểu đồ thay vì cảm tính.

• Nâng cao tính minh bạch cho các khoản chi chung trong gia đình.

• Tạo nền tảng để mở rộng thêm các tính năng tự động hóa, cảnh báo và phân tích nâng cao trong các phiên bản tiếp theo.

Phạm vi và mục tiêu

Phạm vi chức năng của phiên bản hiện tại

• Xác thực người dùng: đăng ký, đăng nhập, quên mật khẩu, đặt lại mật khẩu.

• Quản lý ví tiền cá nhân và ví trong bối cảnh gia đình.

• Quản lý giao dịch theo các loại chi tiêu, thu nhập và chuyển khoản.

• Quản lý mục tiêu tiết kiệm và nạp tiền vào mục tiêu.

• Quản lý nhóm gia đình và quỹ dùng chung.

• Xem báo cáo và biểu đồ tài chính.

• Xem dự báo tài chính.

• Quản lý hồ sơ cá nhân, cài đặt và đa ngôn ngữ.

• Nhận thông báo và đồng bộ một số trạng thái theo thời gian thực.

• Quản trị nền tảng với dashboard admin, thống kê và activity logs.

Phạm vi kỹ thuật

• Frontend web xây dựng bằng React và Vite.

• Backend REST API xây dựng bằng Express và Sequelize.

• Cơ sở dữ liệu PostgreSQL.

• Redis dùng cho cache và healthcheck hệ thống.

• Docker Compose phục vụ luồng chạy tích hợp toàn hệ thống.

• Swagger và Postman phục vụ tài liệu và kiểm thử API.

Những gì chưa nằm trong trọng tâm báo cáo

• Ứng dụng mobile native độc lập.

• Tích hợp ngân hàng hoặc ví điện tử bên thứ ba theo thời gian thực.

• Báo cáo tài chính chuẩn kế toán chuyên nghiệp.

• Cơ chế machine learning hoặc AI dự báo nâng cao ngoài mô hình hiện có.

• Hệ thống phân quyền nhiều tầng cho role staff với luồng quản trị riêng.

Mục tiêu triển khai của đồ án

• Chứng minh hệ thống có thể vận hành end-to-end với frontend, backend, database và cache.

• Thể hiện rõ luồng nghiệp vụ chính qua giao diện thật và tài liệu kỹ thuật.

• Cung cấp deliverable đủ cho bảo vệ cuối kỳ: báo cáo, sơ đồ, ảnh giao diện, slide, tài liệu API và hướng dẫn sử dụng.

• Đảm bảo dự án có thể chạy bằng Docker Compose để người chấm kiểm tra nhanh.

Use Case tổng quan

Actor chính

• Khách: người chưa đăng nhập nhưng có thể tạo tài khoản hoặc khôi phục quyền truy cập.

• Người dùng: người đã đăng nhập và sử dụng các tính năng tài chính cá nhân hoặc gia đình.

• Admin: tài khoản quản trị hệ thống ở cấp nền tảng.

• Hệ thống nền: các thành phần tự động như cron job, realtime notification và các dịch vụ nền hỗ trợ xử lý.

Nhóm use case theo actor

Khách

• Đăng ký tài khoản.

• Đăng nhập.

• Quên mật khẩu.

• Đặt lại mật khẩu.

Người dùng

• Quản lý hồ sơ và cài đặt.

• Quản lý ví tiền.

• Quản lý giao dịch.

• Quản lý mục tiêu tiết kiệm.

• Quản lý gia đình và quỹ chung.

• Xem báo cáo.

• Xem dự báo.

• Nhận thông báo và đồng bộ trạng thái.

Admin

• Xem dashboard quản trị.

• Xem thống kê nền tảng.

• Quản trị người dùng.

• Xem activity logs.

Hệ thống nền

• Đồng bộ dữ liệu realtime qua Socket.IO.

• Chạy cron job cho các tác vụ nền.

• Phục vụ tài liệu Swagger và kiểm tra sức khỏe hệ thống.

Ánh xạ chức năng với mã nguồn hiện tại

Nhóm chức năng	Bằng chứng từ repo
Xác thực	frontend/src/pages/auth/*, backend/routes/authRoutes.js
Ví tiền	frontend/src/pages/Wallets.jsx, backend/routes/walletRoutes.js
Giao dịch	frontend/src/pages/Transactions.jsx, backend/routes/transactionRoutes.js
Mục tiêu	frontend/src/pages/Goals.jsx, backend/routes/goalRoutes.js
Gia đình	frontend/src/pages/Family.jsx, backend/routes/familyRoutes.js
Báo cáo	frontend/src/pages/Reports.jsx, backend/routes/analyticsRoutes.js
Dự báo	frontend/src/pages/Forecast.jsx, backend/routes/forecastRoutes.js
Hồ sơ và cài đặt	frontend/src/pages/Profile.jsx, frontend/src/pages/Settings.jsx
Quản trị hệ thống	frontend/src/pages/AdminDashboard.jsx, backend/routes/adminRoutes.js

Ghi chú mô hình hóa

• Use case được mô hình hóa ở mức tổng quan nghiệp vụ để phục vụ báo cáo thuyết minh.

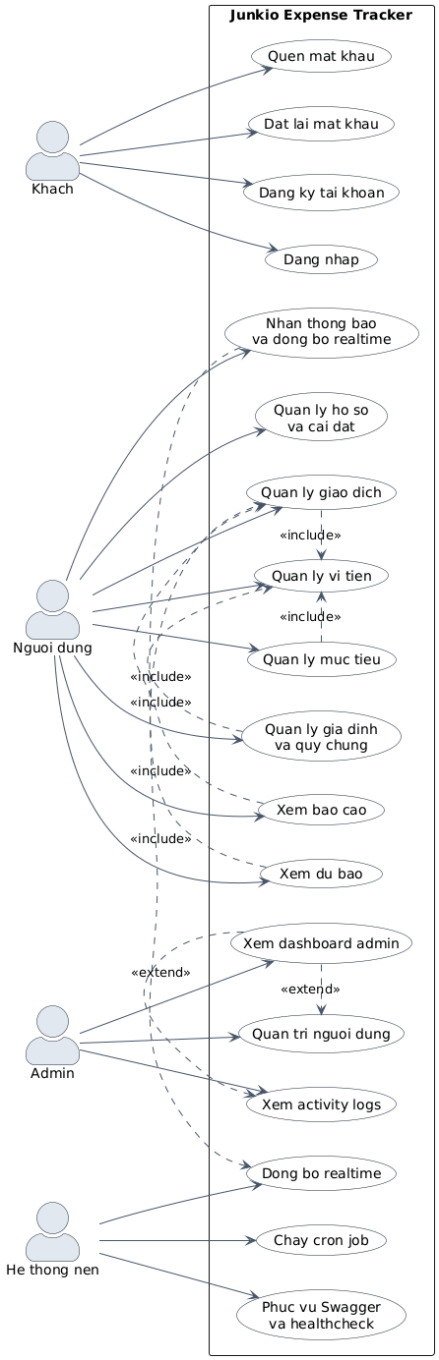
• Các thao tác nhỏ trong cùng một màn hình, ví dụ lọc dữ liệu, tìm kiếm hoặc chuyển tab, được gộp vào use case chính tương ứng.

• Role staff không tách riêng thành actor do chưa có một nhóm use case độc lập đủ khác với admin hoặc người dùng.

Bảng mô tả Use Case

Mã	Use case	Actor chính	Tiền điều kiện	Kết quả sau cùng
UC-01	Đăng ký tài khoản	Khách	Chưa có tài khoản hợp lệ	Tài khoản mới được tạo
UC-02	Đăng nhập	Khách	Có tài khoản hợp lệ	Người dùng vào hệ thống và có phiên đăng nhập

Mã	Use case	Actor chính	Tiền điều kiện	Kết quả sau cùng
UC-03	Quên mật khẩu	Khách	Có email hợp lệ	Hệ thống tạo quy trình khôi phục mật khẩu
UC-04	Đặt lại mật khẩu	Khách	Có token đặt lại còn hiệu lực	Mật khẩu được cập nhật
UC-05	Quản lý hồ sơ và cài đặt	Người dùng	Đã đăng nhập	Thông tin hồ sơ hoặc tùy chọn được cập nhật
UC-06	Quản lý ví tiền	Người dùng	Đã đăng nhập	Ví được tạo, sửa hoặc xem theo bối cảnh cá nhân hoặc gia đình
UC-07	Quản lý giao dịch	Người dùng	Đã đăng nhập và có ít nhất một ví	Giao dịch được thêm, sửa, xem hoặc chuyển khoản
UC-08	Quản lý mục tiêu tiết kiệm	Người dùng	Đã đăng nhập	Mục tiêu được tạo và có thể nạp tiền theo tiến độ
UC-09	Quản lý gia đình và quỹ chung	Người dùng	Đã đăng nhập	Nhóm gia đình hoặc quỹ chung được tạo và sử dụng
UC-10	Xem báo cáo	Người dùng	Đã đăng nhập và có dữ liệu giao dịch	Hệ thống hiển thị thống kê và biểu đồ tài chính
UC-11	Xem dự báo	Người dùng	Đã đăng nhập và có dữ liệu đủ để phân tích	Hệ thống hiển thị dự báo tài chính
UC-12	Nhận thông báo và đồng bộ realtime	Người dùng	Đã đăng nhập	Trạng thái mới được cập nhật lên giao diện
UC-13	Xem dashboard quản trị	Admin	Đã đăng nhập với role admin	Admin xem tổng quan toàn hệ thống
UC-14	Quản trị người dùng	Admin	Đã đăng nhập với role admin	Admin xem, khóa, đổi role hoặc xóa người dùng
UC-15	Xem activity logs	Admin	Đã đăng nhập với role admin	Admin theo dõi nhật ký hoạt động
UC-16	Xử lý nền và cron job	Hệ thống nền	Hệ thống backend đang hoạt động	Các tác vụ định kỳ và đồng bộ tự động được thực thi



Hình UC-1. Biểu đồ Use Case tổng quan của hệ thống

Kiến trúc hệ thống

Góc nhìn tổng thể

Junkio Expense Tracker là hệ thống web full-stack với kiến trúc phân lớp rõ ràng giữa frontend, backend API, cơ sở dữ liệu và hạ tầng phụ trợ. Frontend đảm nhiệm giao diện và trải nghiệm người dùng. Backend đảm nhiệm xác thực, xử lý nghiệp vụ, kết nối cơ sở dữ liệu, realtime và tài liệu API. PostgreSQL lưu trữ dữ liệu nghiệp vụ, còn Redis hỗ trợ kết nối và healthcheck.

Thành phần chính

Thành phần	Công nghệ	Vai trò
Frontend Web	React 19, Vite, Redux Toolkit, Tailwind CSS	Hiển thị giao diện, quản lý state, gọi API, vẽ biểu đồ và xuất báo cáo
Backend API	Node.js, Express, Sequelize	Xử lý nghiệp vụ, phân quyền, truy cập dữ liệu, cung cấp REST API
Database	PostgreSQL	Lưu trữ dữ liệu người dùng, ví, giao dịch, mục tiêu, gia đình, ngân sách
Cache và hỗ trợ hệ thống	Redis	Hỗ trợ dịch vụ nền và kiểm tra sức khỏe hệ thống
Realtime	Socket.IO	Đồng bộ một số sự kiện và thông báo theo thời gian thực
Tài liệu API	Swagger UI, Postman	Tài liệu hóa và kiểm thử endpoint
Triển khai	Docker Compose, Nginx	Đóng gói và chạy tích hợp frontend, backend, PostgreSQL và Redis

Cấu trúc backend theo trách nhiệm

• Routes: gom nhóm endpoint theo module như auth, wallets, transactions, families, goals, analytics, admin, forecast.

• Controllers: tiếp nhận request và điều phối xử lý.

• Services: chứa các xử lý nền và nghiệp vụ hỗ trợ như email, cron job, recurring execution.

• Models: mô hình dữ liệu Sequelize.

• Middleware: auth, role, upload, response, audit.

Cấu trúc frontend theo trách nhiệm

• Pages: Dashboard, Transactions, Wallets, Goals, Family, Reports, Forecast, Settings, Profile, AdminDashboard.

• Auth layer: PrivateRoutes và AdminRoute.

• State management: Redux Toolkit cho auth, analytics, wallets, families, goals, recurring, transactions.

• UI utilities: i18n, charts, export PDF/CSV/XLSX.

Data flow tiêu biểu

Luồng đăng nhập

1. Người dùng mở trang đăng nhập từ frontend. 2. Frontend gửi thông tin xác thực tới backend qua authRoutes. 3. Backend xác thực tài khoản, tạo token và trả về phản hồi. 4. Frontend lưu token, lấy thông tin người dùng hiện tại và điều hướng về giao diện chính. 5. Sau khi đăng nhập, frontend tiếp tục nạp wallets, families, goals, recurring, transactions và analytics.

Luồng tạo và xem giao dịch

1. Người dùng mở màn hình giao dịch và tạo một giao dịch mới. 2. Frontend gửi request tới transactionRoutes. 3. Backend kiểm tra quyền, dữ liệu đầu vào và ví liên quan. 4. Sequelize ghi dữ liệu vào PostgreSQL. 5. Frontend cập nhật danh sách giao dịch, dashboard analytics và report analytics. 6. Một số trạng thái có thể được đồng bộ lại qua socket hoặc thông báo trên giao diện.

Triển khai Docker

• web: frontend chạy sau khi build và phục vụ qua Nginx.

• api: backend Express chạy trên cổng 5000.

• db: PostgreSQL dùng volume riêng cho dữ liệu.

• redis: Redis phục vụ thành phần phụ trợ.

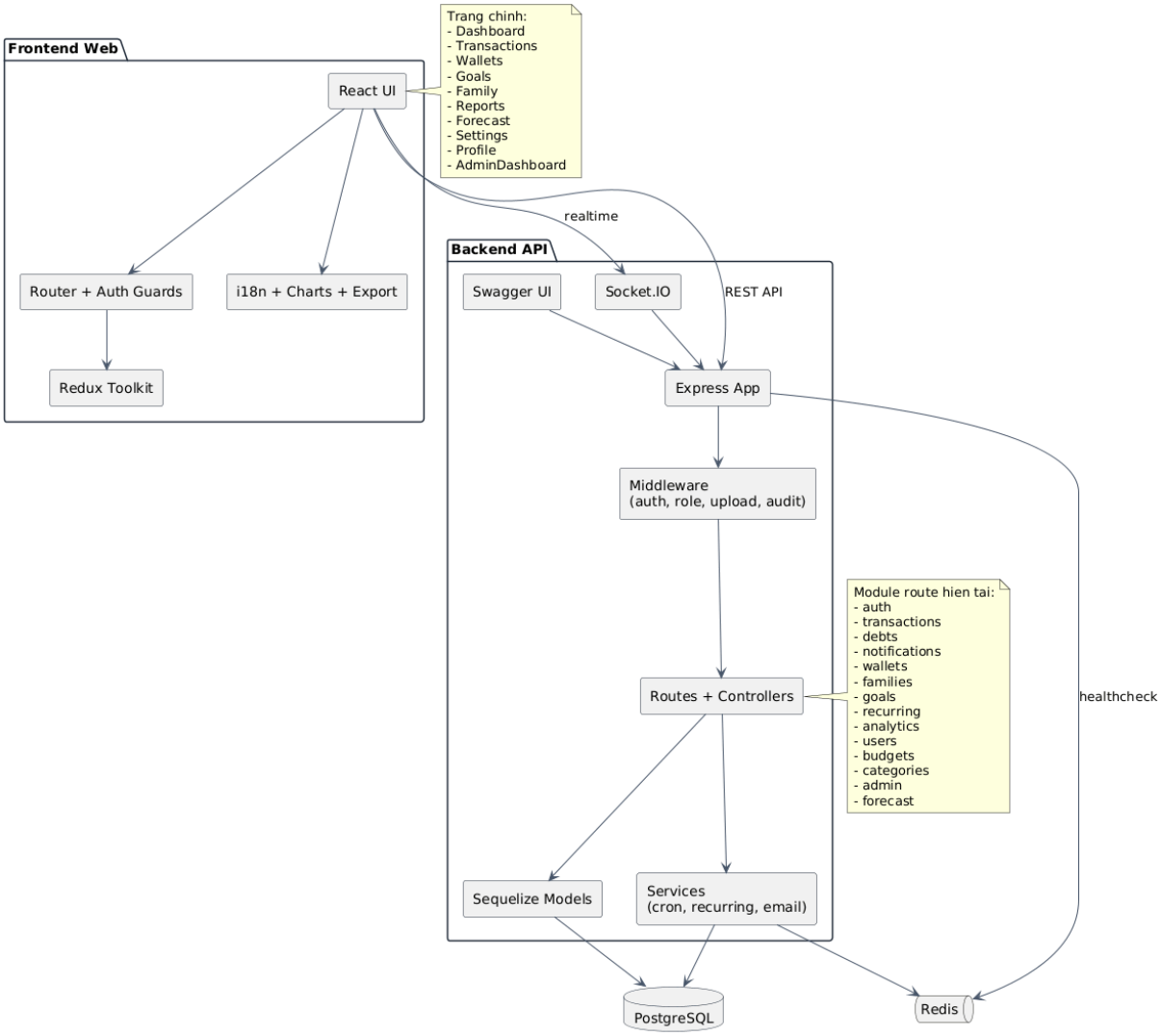
• api phụ thuộc db và redis, còn web phụ thuộc api và redis.

Ghi chú kiến trúc

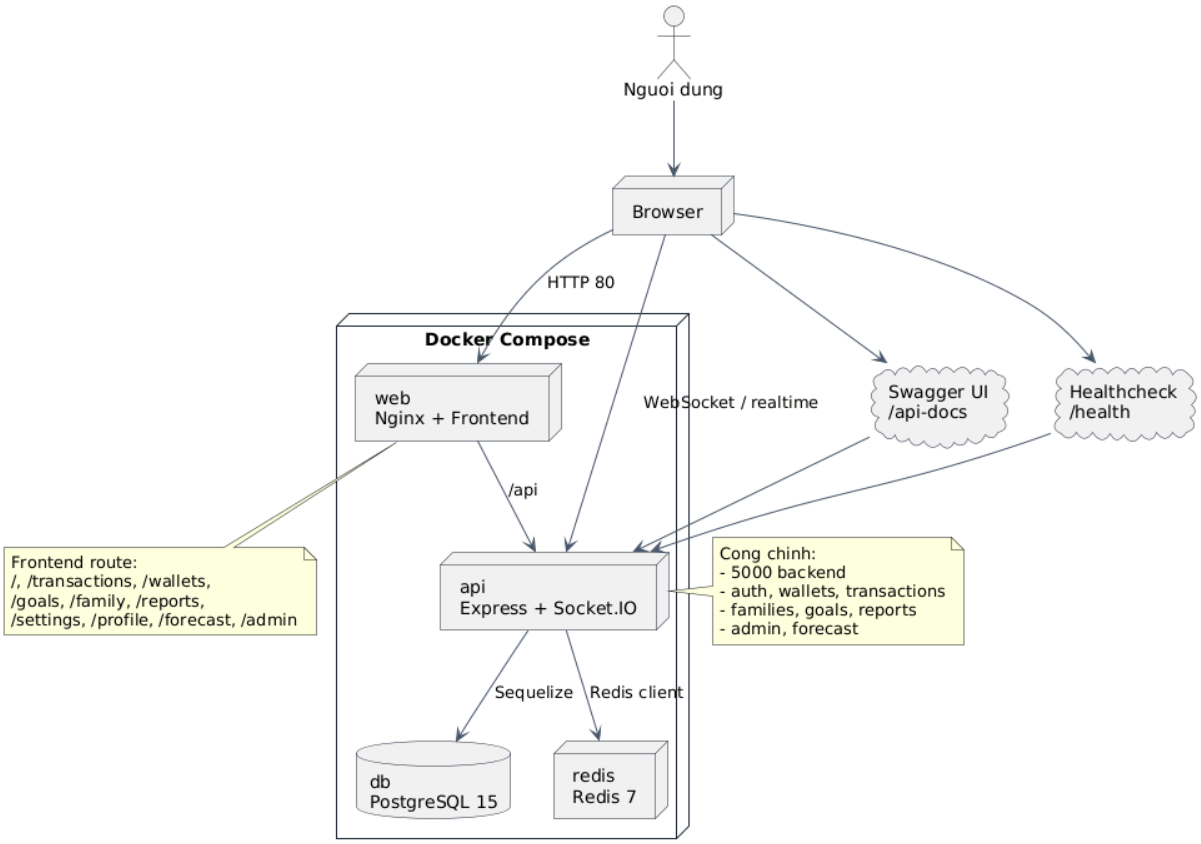
• Kiến trúc hiện tại ưu tiên tính đầy đủ tính năng và khả năng chạy end-to-end cho môi trường đồ án.

• Swagger và Postman là phần quan trọng của tài liệu kỹ thuật, không phải thành phần nghiệp vụ nhưng có vai trò lớn trong kiểm thử và nghiệm thu.

• Role admin được tách rõ ở frontend route và backend route bằng middleware phân quyền.



Hình KT-1. Sơ đồ kiến trúc thành phần



Hình KT-2. Sơ đồ kiến trúc triển khai

Danh mục ảnh giao diện

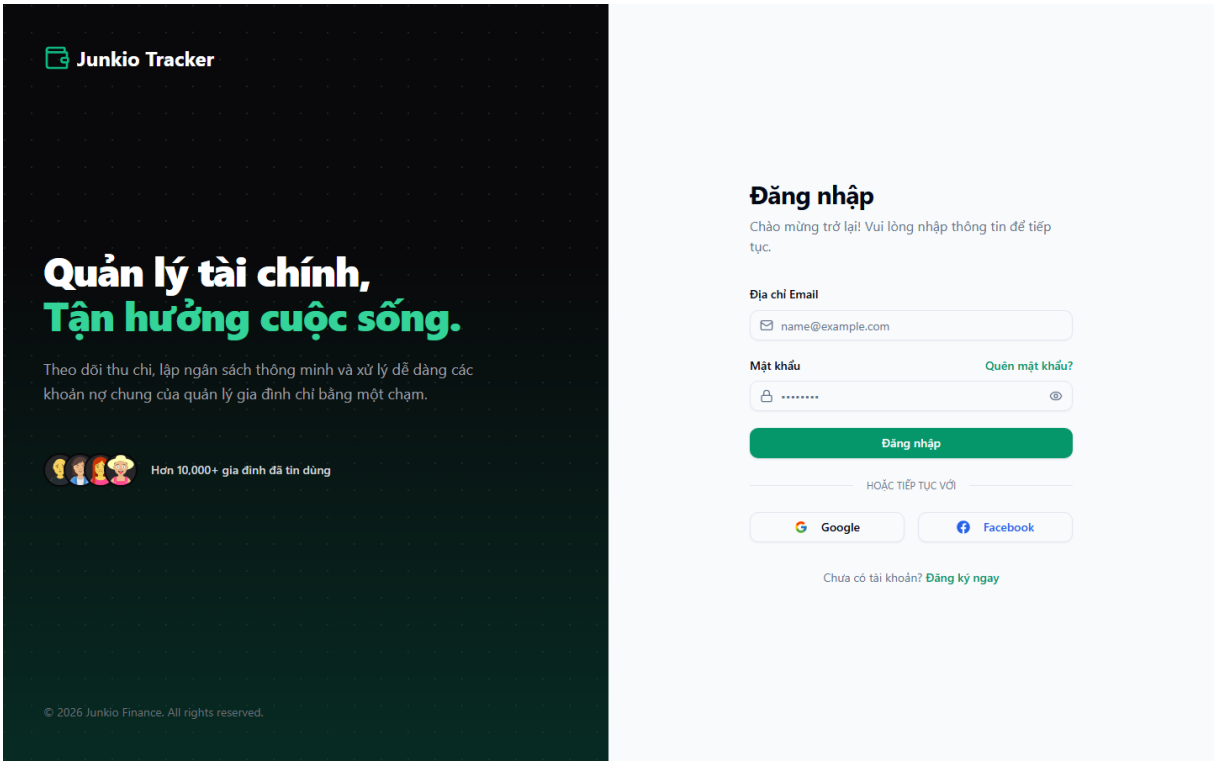
Ảnh người dùng

Tên file	Mô tả hiển thị
01-dang-nhap.png	Màn hình đăng nhập của hệ thống
02-dashboard-tong-quan.png	Trang tổng quan tài chính sau khi đăng nhập
03-giao-dich.png	Màn hình quản lý giao dịch
04-vi-tien.png	Màn hình quản lý ví tiền
05-muc-tieu-tiet-kiem.png	Màn hình quản lý mục tiêu tiết kiệm
06-gia-dinh-va-quy-chung.png	Màn hình nhóm gia đình và quỹ chung
07-bao-cao.png	Màn hình báo cáo và biểu đồ tài chính
08-du-bao.png	Màn hình dự báo tài chính
09-cai-dat.png	Màn hình cài đặt người dùng
10-ho-so-nguoi-dung.png	Màn hình hồ sơ người dùng

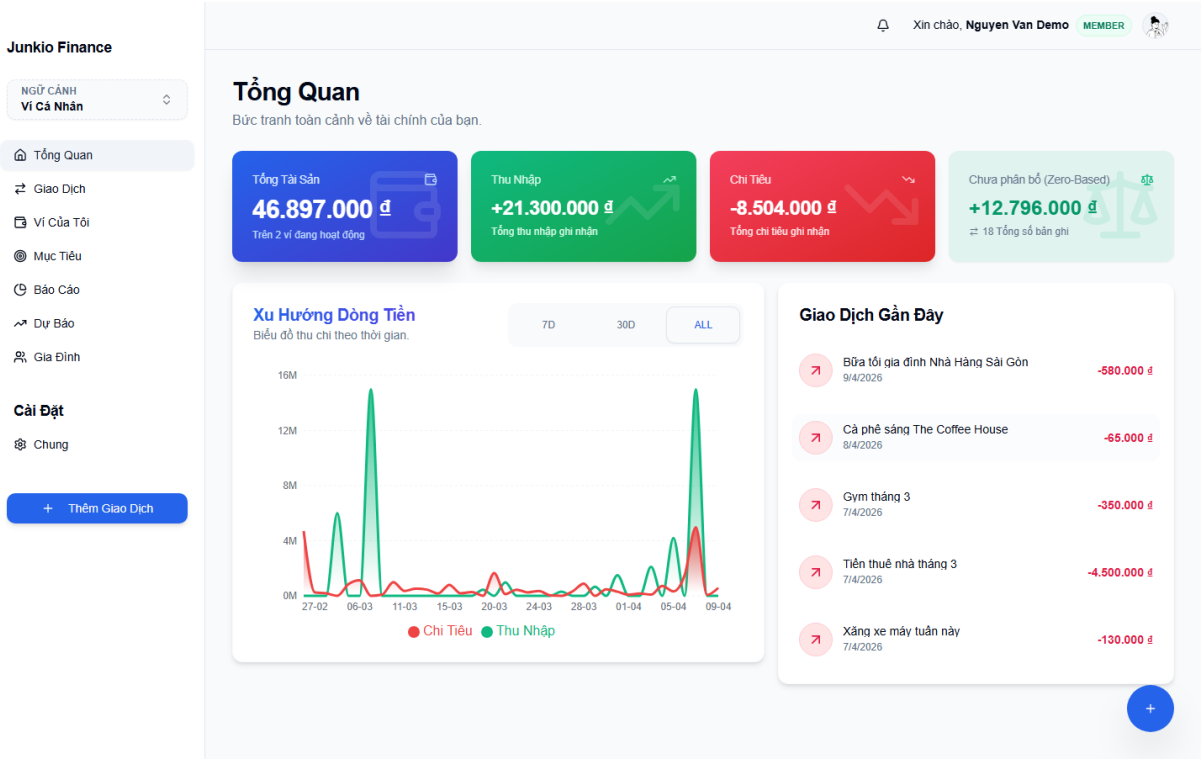
Ảnh admin

Tên file	Mô tả hiển thị
admin-01-tong-quan-he-thong.png	Dashboard quản trị tổng quan hệ thống
admin-02-quan-ly-nguoi-dung.png	Khu vực quản lý danh sách người dùng
admin-03-chi-tiet-nguoi-dung.png	Chi tiết một người dùng trong khu vực quản trị
admin-04-nhat-ky-hoat-dong.png	Tab nhật ký hoạt động của admin

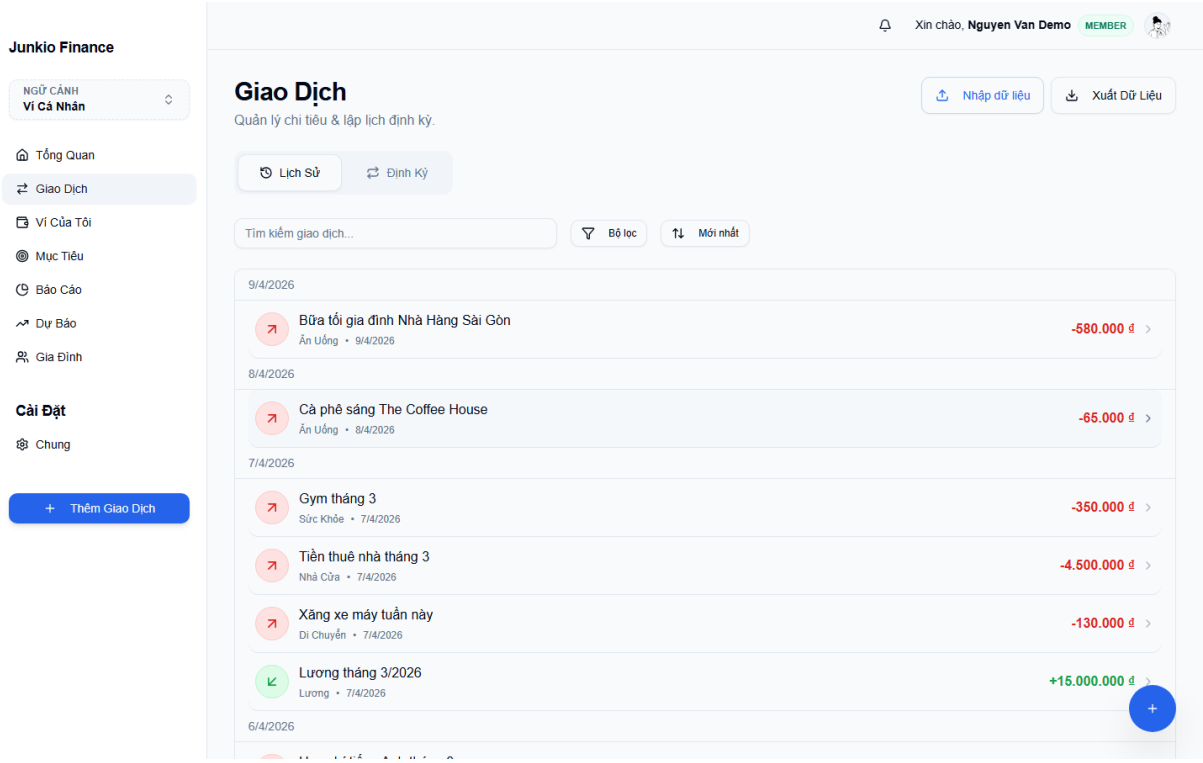
Bộ ảnh giao diện người dùng



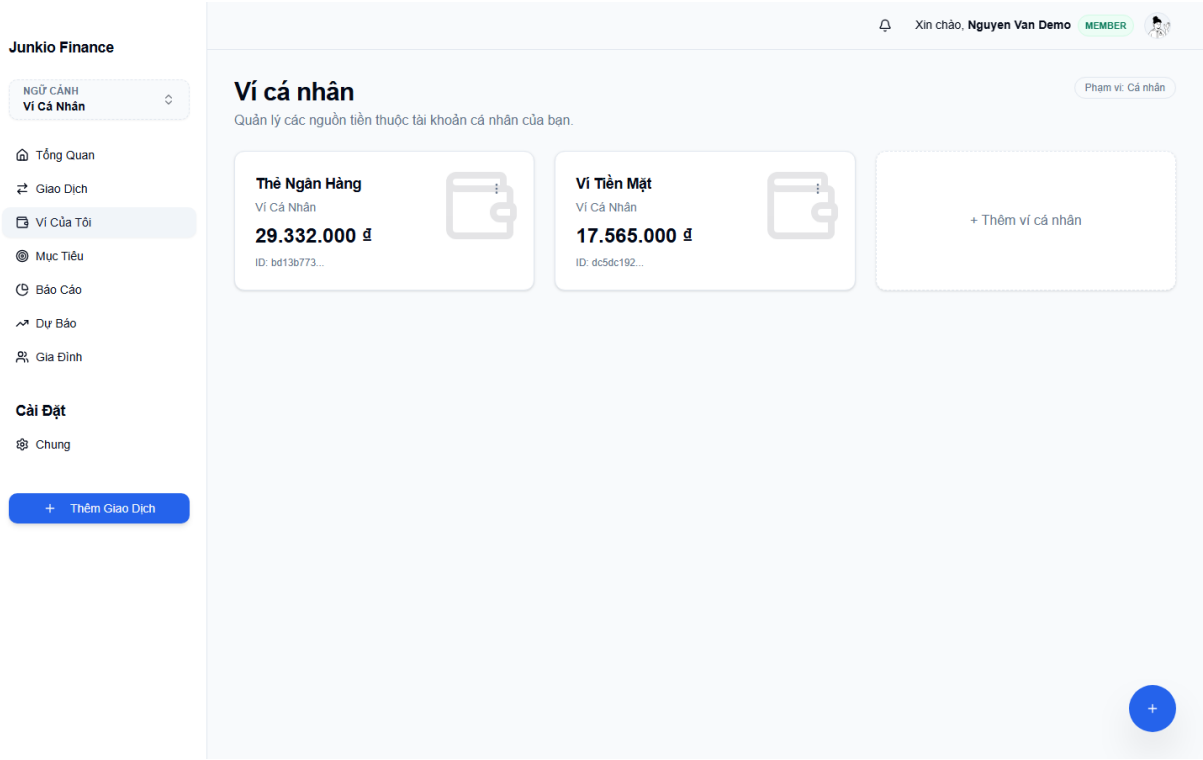
Hình 1. Màn hình đăng nhập hệ thống



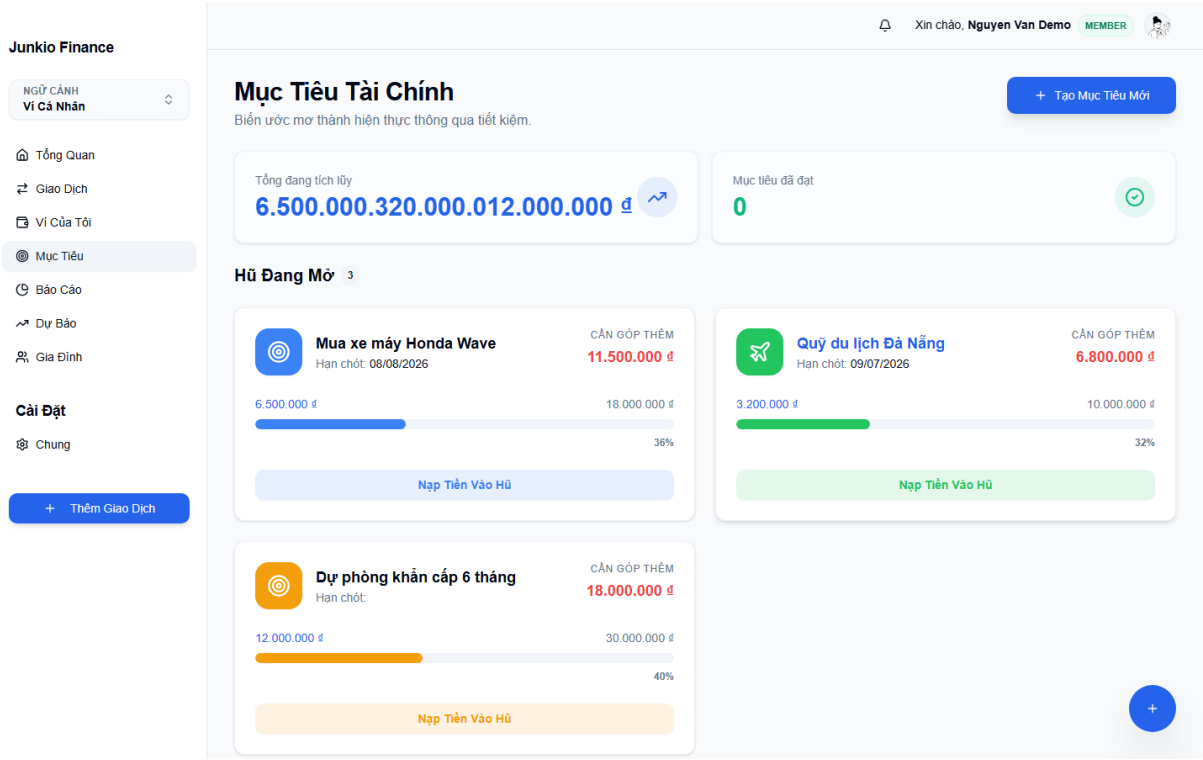
Hình 2. Trang tổng quan tài chính của người dùng



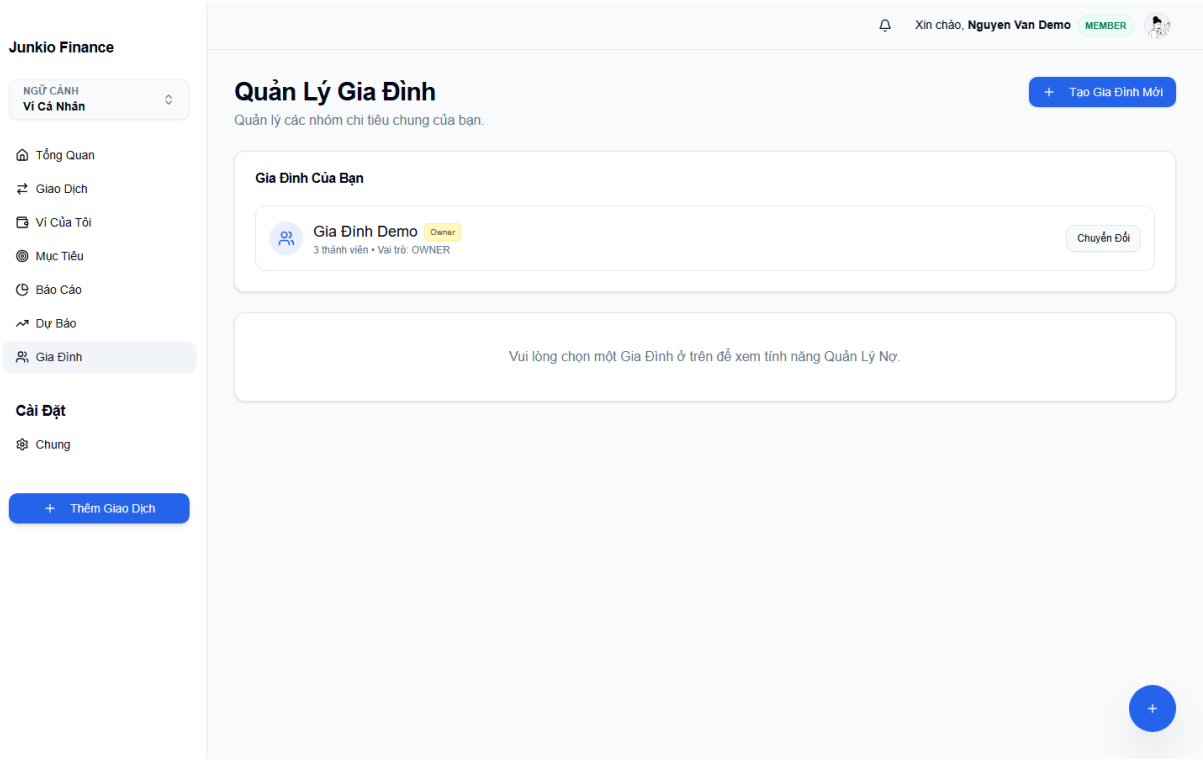
Hình 3. Màn hình quản lý giao dịch



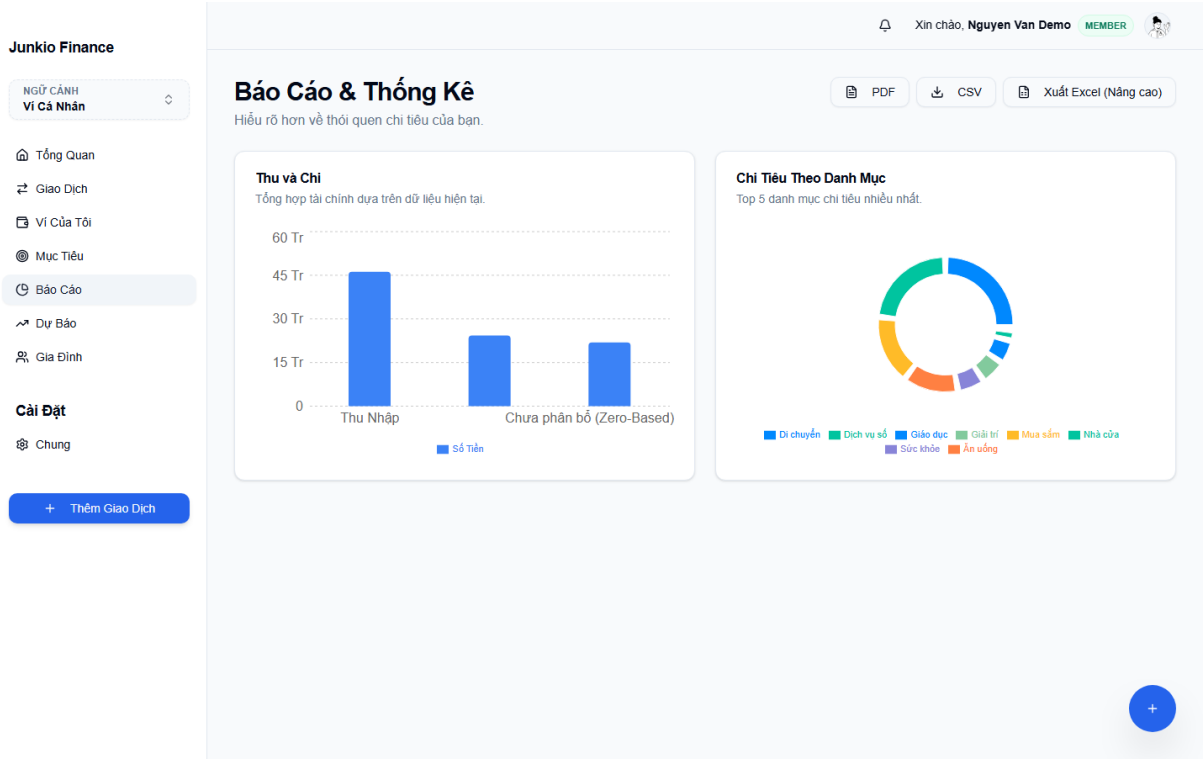
Hình 4. Màn hình quản lý ví tiền



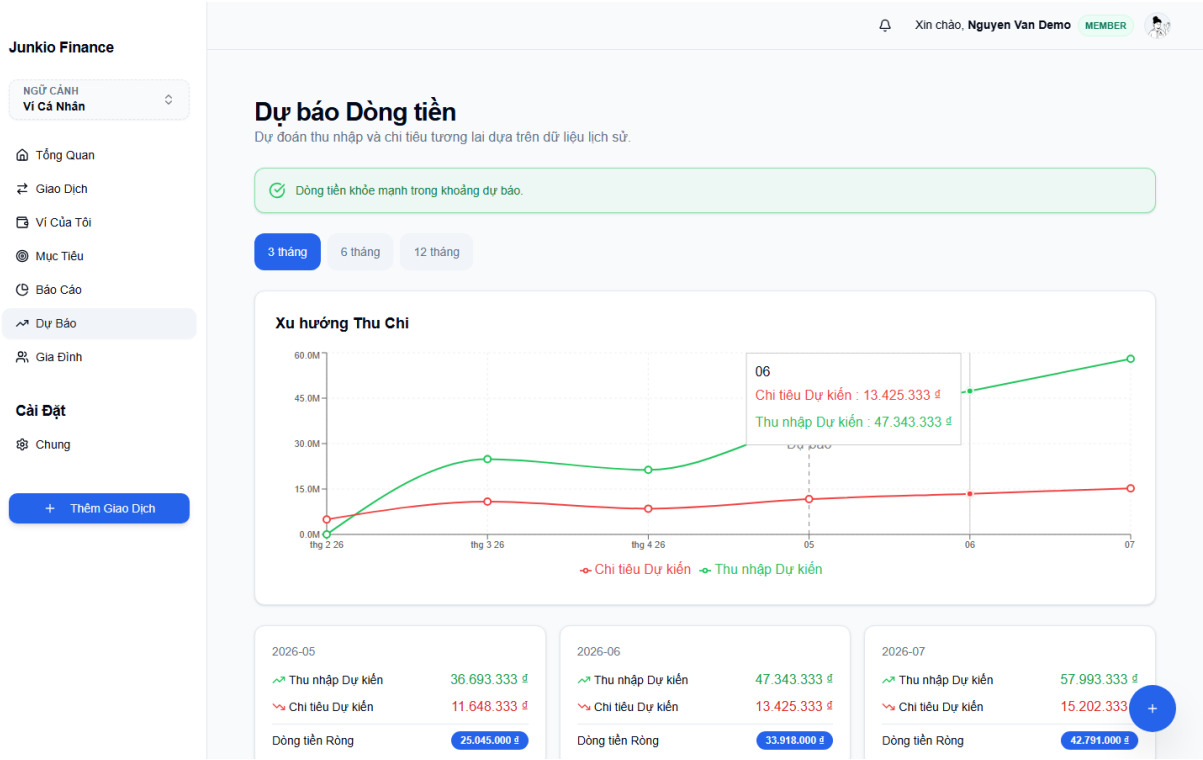
Hình 5. Màn hình quản lý mục tiêu tiết kiệm



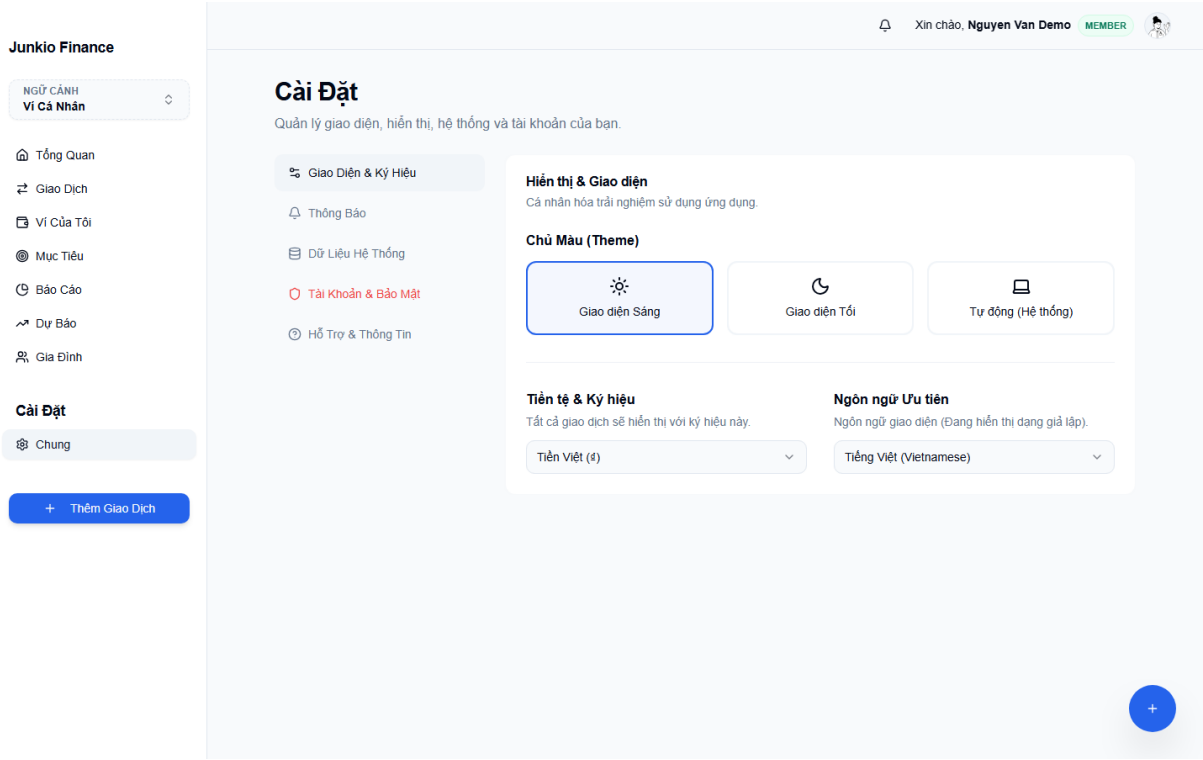
Hình 6. Màn hình gia đình và quỹ chung



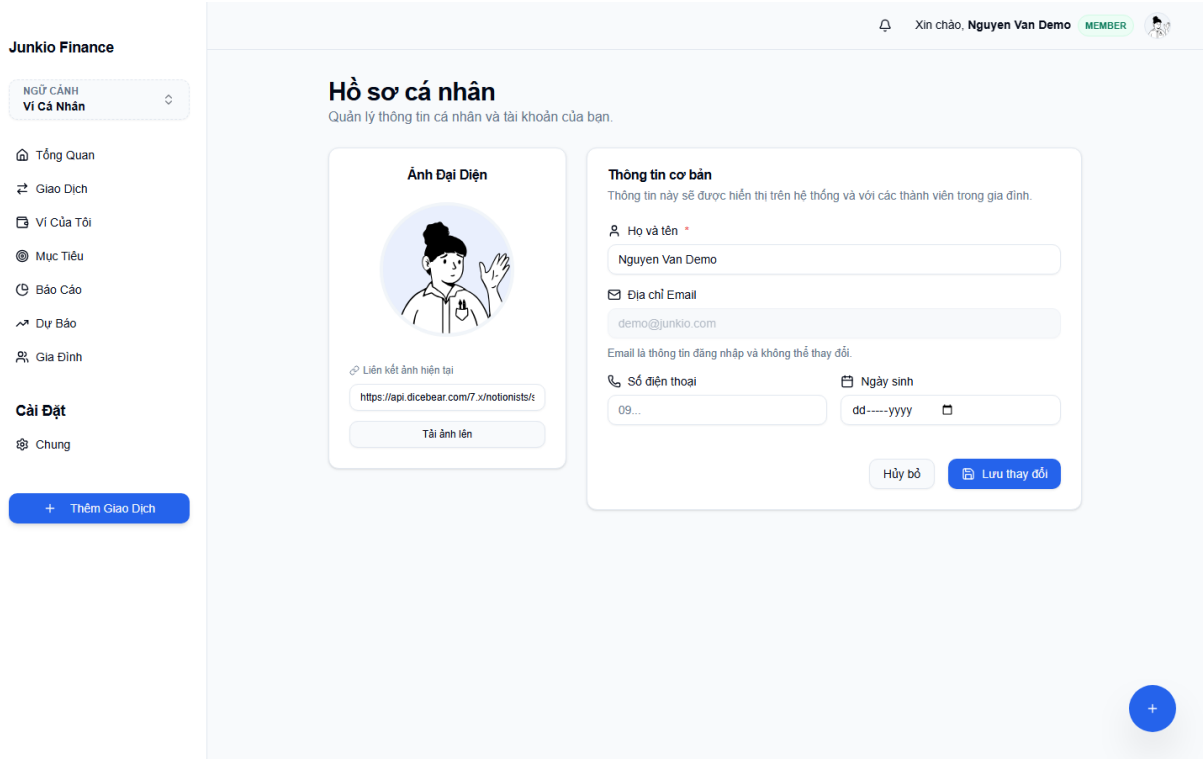
Hình 7. Màn hình báo cáo và thống kê



Hình 8. Màn hình dự báo tài chính

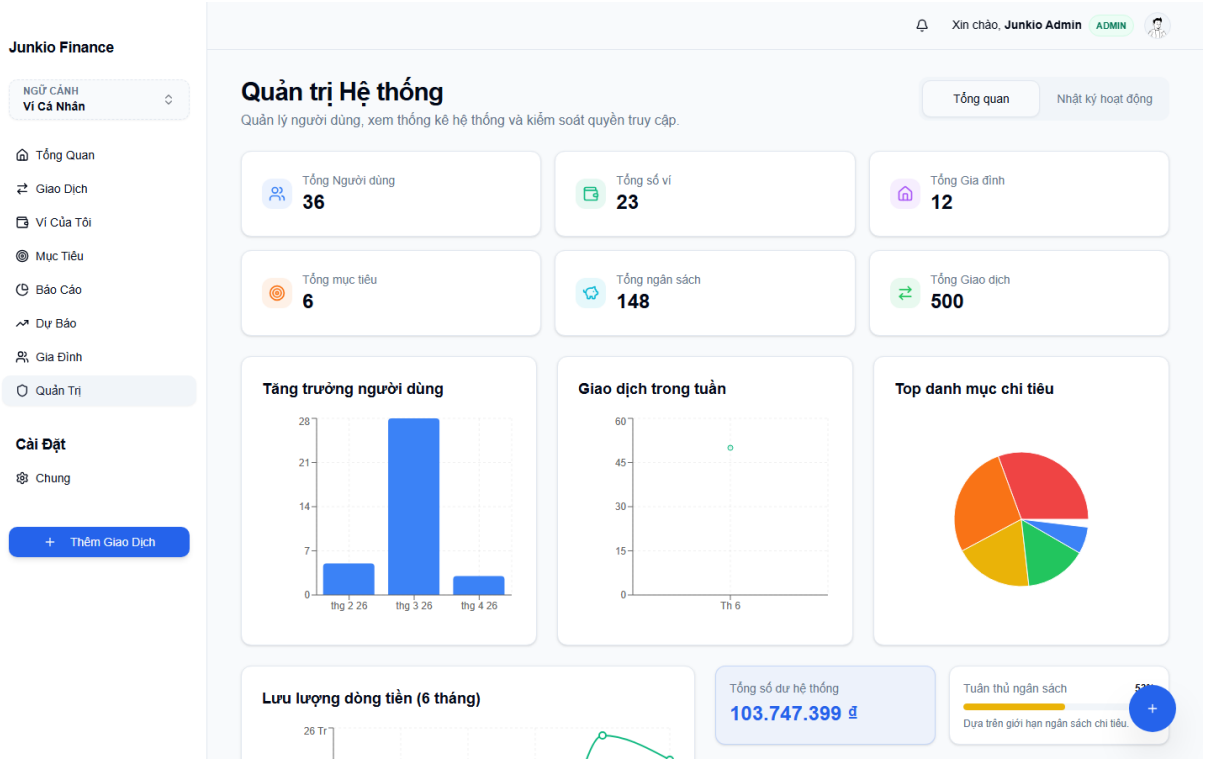


Hình 9. Màn hình cài đặt người dùng

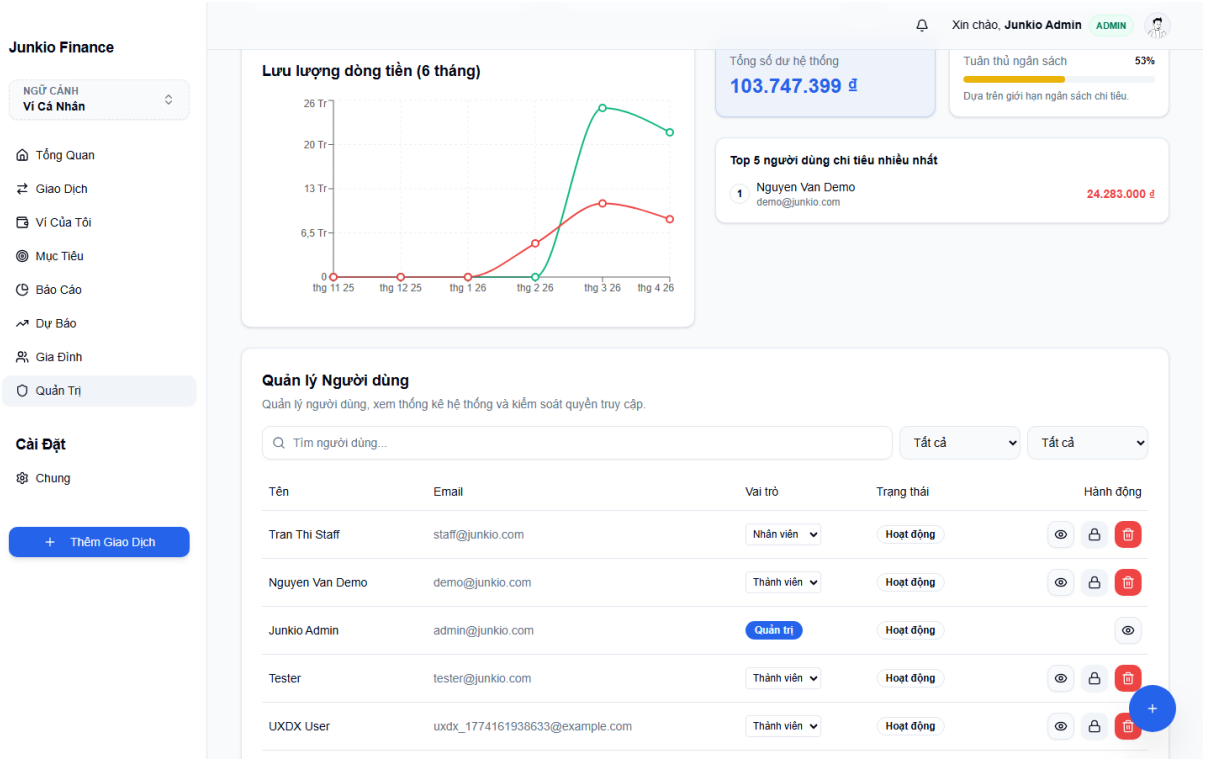


Hình 10. Màn hình hồ sơ người dùng

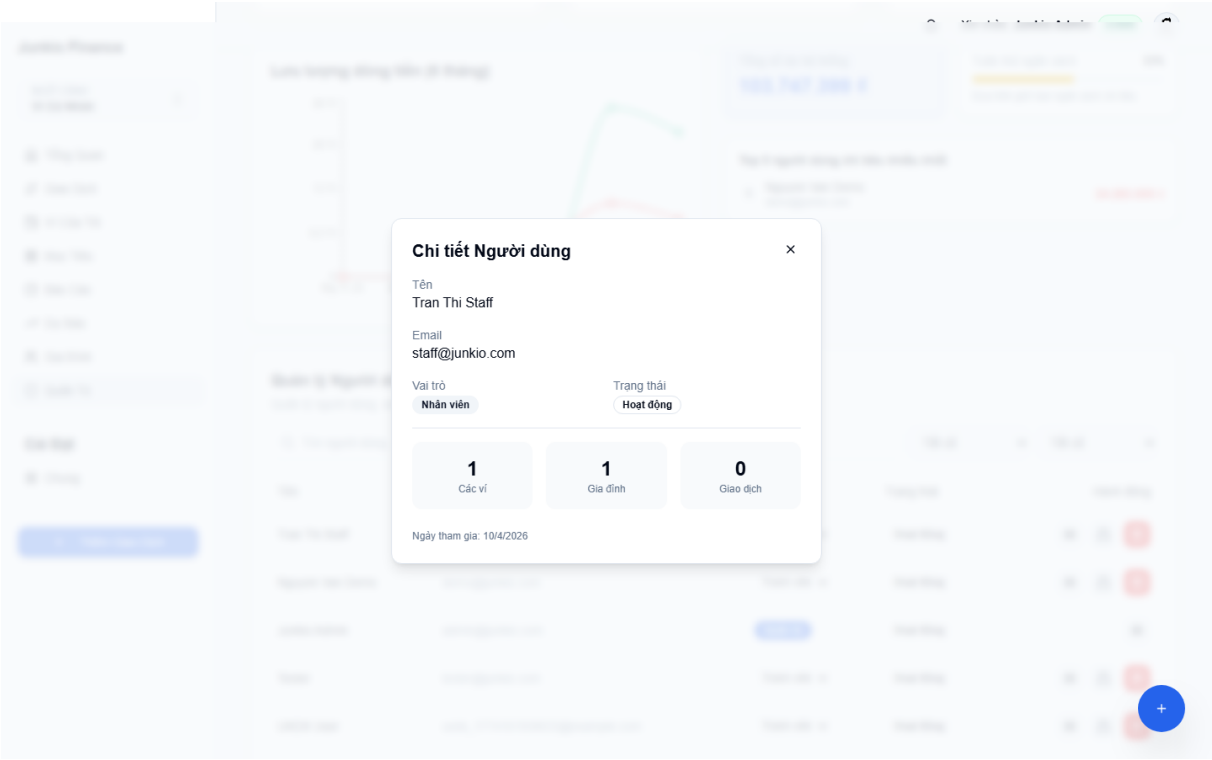
Bộ ảnh giao diện admin



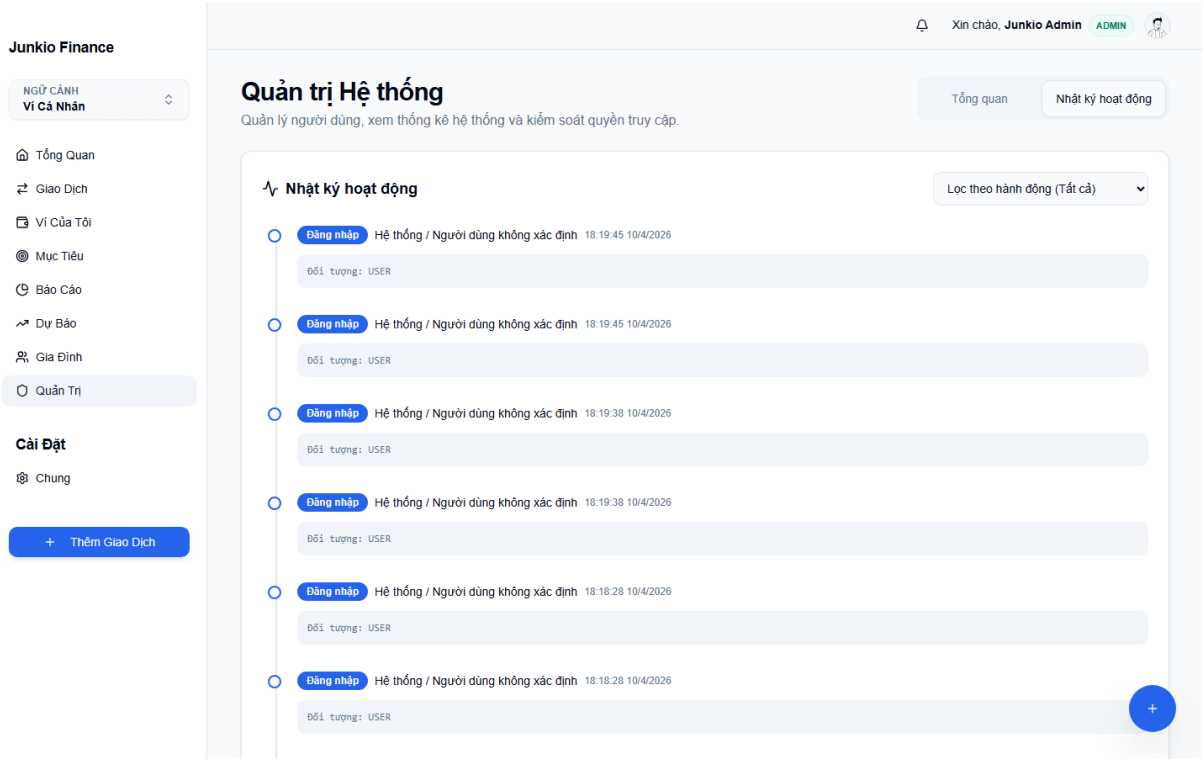
Hình 11. Dashboard quản trị tổng quan hệ thống



Hình 12. Khu vực quản lý người dùng của admin



Hình 13. Chi tiết một người dùng trong khu vực quản trị



Hình 14. Nhật ký hoạt động của hệ thống

Hướng dẫn cài đặt và sử dụng

1. Yêu cầu môi trường

• Docker Desktop hoặc Docker Engine có hỗ trợ Docker Compose.

• Trình duyệt web hiện đại.

• Nếu chạy local dev: Node.js 18+, PostgreSQL và Redis.

2. Chạy hệ thống bằng Docker Compose

Đứng tại thư mục gốc của dự án và chạy:

```
docker compose up -d --build
docker compose ps
```

Sau khi các service hoạt động:

• Giao diện web: http://localhost

• Backend API: http://localhost:5000

• Swagger UI: http://localhost:5000/api-docs

• Healthcheck: http://localhost:5000/health

3. Quy trình sử dụng nhanh cho người chấm

Đăng nhập người dùng thường

1. Truy cập http://localhost. 2. Mở màn hình đăng nhập. 3. Dùng tài khoản demo. 4. Sau khi đăng nhập, kiểm tra lần lượt các trang: Dashboard, Giao dịch, Ví tiền, Mục tiêu, Gia đình, Báo cáo, Dự báo, Cài đặt, Hồ sơ.

Đăng nhập admin

1. Đăng xuất tài khoản hiện tại nếu cần. 2. Đăng nhập bằng tài khoản admin. 3. Truy cập /admin. 4. Kiểm tra các khu vực chính:

• dashboard tổng quan

• quản lý người dùng

• activity logs

4. Luồng thao tác chính của người dùng

Quản lý ví

• Vào trang Ví tiền.

• Tạo ví mới theo loại tiền mặt hoặc ngân hàng.

• Kiểm tra số dư và bối cảnh cá nhân hoặc gia đình.

Quản lý giao dịch

• Vào trang Giao dịch.

• Thêm giao dịch thu, chi hoặc chuyển khoản.

• Kiểm tra giao dịch xuất hiện trong danh sách và dashboard.

Quản lý mục tiêu

• Vào trang Mục tiêu.

• Tạo mục tiêu tiết kiệm mới.

• Nạp tiền vào mục tiêu từ một ví cá nhân.

Quản lý gia đình

• Vào trang Gia đình.

• Tạo nhóm gia đình hoặc quỹ chung.

• Chuyển bối cảnh làm việc giữa cá nhân và gia đình.

Xem báo cáo và dự báo

• Vào trang Báo cáo để xem biểu đồ và thống kê.

• Vào trang Dự báo để xem xu hướng tài chính ước lượng.

5. Tài liệu API và kiểm tra hệ thống

• Swagger UI dùng để xem và thử API trực tiếp.

• Bộ Postman có sẵn trong docx/07-tham-chieu/postman/.

• Healthcheck giúp xác nhận backend, PostgreSQL và Redis đang hoạt động.

6. Phụ lục local development

Nếu không chạy Docker Compose:

```
cd backend
npm install
npm run dev
```

Mở terminal mới:

```
cd frontend
npm install
npm run dev
```

Backend mặc định chạy ở http://localhost:5000, frontend local ở http://localhost:5173.

Tài khoản demo

Tài khoản có sẵn

Vai trò	Email	Mật khẩu	Mục đích
Admin hệ thống	admin@junkio.com	admin123	Kiểm tra màn hình quản trị
Staff	staff@junkio.com	staff123	Vai trò dữ liệu phục vụ kiểm tra phân quyền
Người dùng thường	demo@junkio.com	demo123	Kiểm tra các chức năng nghiệp vụ chính

Nguồn dữ liệu

• Dữ liệu demo được sinh từ backend/scripts/seed-demo.js.

• Khi chạy bằng Docker Compose với AUTO_SEED=true, backend sẽ tự chạy seed ở thời điểm khởi động.

• Bộ dữ liệu seed bao gồm người dùng mẫu, ví, categories và một số giao dịch minh họa.

Tài liệu tham chiếu

Swagger snapshot

• docx\07-tham-chieu\swagger\api-docs.md

• docx\07-tham-chieu\swagger\swagger.json

• docx\07-tham-chieu\swagger\swagger.yaml

Postman snapshot

• docx\07-tham-chieu\postman\Junkio.postman_collection.json

• docx\07-tham-chieu\postman\Junkio.postman_environment.json

ERD và CSDL

• docx\07-tham-chieu\erd\Diagram.svg

• docx\07-tham-chieu\erd\Junkio_CSDL_dbdiagram_ready.dbml

Slide

• docx\07-tham-chieu\slide\junkio.pptx

Legacy

• docx\legacy\BAOCAO3th4.docx